

**UNIVERSIDAD DEL VALLE DE
GUATEMALA**

Facultad de Ingeniería

Departamento de Ciencias de la
Computación

**CC3086 – Programación de
Microprocesadores**

Ciclo 1 – 2026

Sección: 10

Integrantes:

Daniel Rivas, Clever



PROYECTO #1

Programación en C para el control de circuitos con STM32

Fecha de entrega: 13 – 17 de abril de 2026

-Resumen del proyecto

El Proyecto 1 de tiene como objetivo desarrollar un sistema embebido usando una placa STM32 Nucleo, el lenguaje C y los periféricos de entrada o salida básicos del microcontrolador. El sistema controla ocho LEDs conectados al puerto GPIO C mediante una secuencia cíclica de patrones. La temporización de la secuencia se basa en el contador del sistema SysTick y es no bloqueante, de este modo, la microcontroladora sigue ejecutando código mientras espera el siguiente cambio de estado en vez de no hacer nada y esperar.

Hay 2 botones que permiten aumentar o disminuir la velocidad de la secuencia. El período base inicial es 1,5 s y existen tres niveles de velocidad que son estos 1,5 s, 800 ms y 300 ms. Al presionar los botones se modifica la velocidad y un conjunto de LEDs indicadores señala el nivel actual. La secuencia siempre comienza desde el primer estado y la velocidad más baja cuando el sistema se reinicia. La interacción con los botones se gestiona mediante interrupciones externas EXTI y un algoritmo de debounce por software.

– Explicación general del sistema

El sistema se desarrolla en lenguaje C utilizando la HAL de STMicroelectronics. La lógica general es la siguiente:

1. **Secuencia de LEDs.** Se define un arreglo de diez patrones de 8 bits (secuencia), donde cada bit representa el encendido o apagado de uno de los ocho LEDs. El programa recorre estos valores para generar la secuencia de patrones.
2. **Temporización no bloqueante.** Para evitar retrasos bloqueantes, el programa utiliza la función `HAL_GetTick()`, la cual devuelve un valor de tick en milisegundos. Almacena el valor del tick cada vez que cambia de estado (`last_tick`) y compara el tiempo transcurrido con el retardo (`current_delay`). Cuando el tiempo transcurrido es mayor o igual al retardo, el programa avanza al siguiente patrón sin detener la ejecución.
3. **Control de velocidad.** Se define un arreglo `speed_delays` con tres valores en milisegundos que son 1500 ms, 800 ms y 300 ms. Una variable `current_speed_level` actúa como índice para seleccionar el período actual. Dos botones conectados a entradas configuradas como interrupciones (EXTI) permiten aumentar (PC13) o disminuir (PB8)

la velocidad. Un debounce por software ignora cualquier interrupción que ocurra antes de 200 ms desde la última pulsación.

4. **Gestión de botones con interrupciones.** Las entradas están configuradas para generar interrupciones en flanco de subida. Cuando un botón cambia de estado, el núcleo detiene la ejecución del programa principal y ejecuta la rutina de servicio a la interrupción (ISR). Esta rutina actualiza la velocidad y regresa al programa principal. Los artículos técnicos sobre interrupciones explican que, al ocurrir una interrupción, la CPU suspende temporalmente la ejecución secuencial y atiende la función ISR; después vuelve al flujo normal. Las interrupciones permiten que el sistema sea **reactivo** y evitan la comprobación constante del estado de los botones.
5. **Indicador visual de velocidad.** Tres LEDs adicionales muestran el nivel de velocidad. Según el valor de `current_speed_level`, la función `update_speed_leds()` enciende el LED correspondiente (en los pines PA7, PA6 y PA5) y apaga los otros dos.

– Explicación del código por bloques

1. Arreglos

- **secuencia[10]:** contiene diez valores de 8 bits (0xE0, 0x70, 0x38, 0x1C, 0x0E, 0x07, 0x0E, 0x1C, 0x38 y 0x70). Cada bit corresponde a uno de los ocho LEDs de salida. El programa recorre estos valores para generar la secuencia de patrones.
- **speed_delays[3]:** almacena tres retardo en milisegundos: 1500 ms, 800 ms y 300 ms. Estos valores corresponden a las velocidades lenta, media y rápida respectivamente, cumpliendo con el requerimiento de al menos tres niveles de velocidad.

2. Variables globales

- **volatile uint8_t current_speed_level:** indica el nivel de velocidad actual (0, 1 o 2). Se declara volatile porque su valor puede cambiar dentro de la rutina de interrupción.
- **volatile uint32_t current_delay:** almacena el retardo actual en milisegundos, tomado de `speed_delays[current_speed_level]`.

- **uint8_t state_index:** índice del patrón actual dentro de secuencia. Se incrementa cada vez que se actualiza el estado y vuelve a cero al final del arreglo.
- **uint32_t last_tick:** guarda el valor de HAL_GetTick() en el momento del último cambio de estado. Se usa para calcular el tiempo transcurrido.

3. Funciones

3.1 update_speed_leds()

Esta función enciende uno de los tres LEDs indicadores de velocidad y apaga los otros dos. Llama a HAL_GPIO_WritePin() para escribir un valor de 0 o 1 en los pines PA7, PA6 y PA5. La función HAL_GPIO_WritePin() establece o limpia los bits de un puerto utilizando los registros BSRR/BRR de la HAL, permitiendo modificaciones atómicas de un pin.

3.2 HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)

Esta rutina se ejecuta cada vez que una entrada configurada como interrupción (PC13 o PB8) detecta un flanco ascendente. Implementa un debounce, compara el tiempo transcurrido desde la última interrupción y, si han pasado más de 200 ms, interpreta la pulsación. Si el pin de la interrupción es PC13, incrementa el nivel de velocidad (hasta un máximo de 2). Si el pin es PB8, decrementa el nivel (hasta un mínimo de 0). A continuación actualiza current_delay con el nuevo valor del arreglo speed_delays y llama a update_speed_leds() para actualizar los LEDs indicadores. Al finalizar, almacena el tiempo de la interrupción en last_interrupt_time.

3.3 Inicialización en main()

En el archivo main.c se inicializan el reloj (SystemClock_Config), las GPIO (MX_GPIO_Init) y la HAL. Después de la configuración, el programa inicializa las variables globales:

3.4 Lazo principal (while(1))

Dentro del bucle infinito del programa se ejecuta la lógica principal. Se evalúa si el tiempo transcurrido (HAL_GetTick() - last_tick) es mayor o igual a current_delay. Cuando se cumple, se actualiza last_tick, se incrementa state_index (reiniciándolo a cero al llegar al final) y se obtienen los ocho bits correspondientes del arreglo secuencia. Se escribe cada bit en el pin correspondiente, actualizando así la salida de los LEDs. De este modo se logra una

temporización no bloqueante, ya que el sistema no se detiene mediante retardos fijos sino que controla el tiempo transcurrido mediante el contador de la HAL

-Tabla de variables

Tabla 1: variables del código

Variable	Tipo	Valor inicial	Función dentro del sistema
secuencia	const uint8_t[10]]	{0xE0, 0x70, 0x38, 0x1C, 0x0E, 0x07, 0x0E, 0x1C, 0x38, 0x70}	Define la secuencia de diez patrones binarios para los ocho LEDs de salida.
speed_delays	const uint32_t[3]]	{1500, 800, 300}	Tabla de períodos (ms) para los tres niveles de velocidad.
current_speed_level	volatile uint8_t	0	Índice de la velocidad actual; permite seleccionar la posición en speed_delays. Es volatile porque cambia dentro de la ISR.
current_delay	volatile uint32_t	1500	Retardo actual entre cambios de estado, en milisegundos.
state_index	uint8_t	0	Índice del patrón actual de la secuencia; se

			incrementa y reinicia al llegar al final.
last_tick	uint32_t	0	Valor del tick del sistema (HAL_GetTick()) en el último cambio de estado.
last_interrupt_time	static uint32_t	0 (local a la ISR)	Tiempo de la última interrupción aceptada; se usa para implementar el debounce de 200 ms.

Nota: La tabla 1 muestra todas las variables del código

– Tabla de funciones

Tabla 2: funciones del código

Función	Propósito	Entradas	Salidas	Descripción
update_speed_leds()	Actualizar los LEDs indicadores de velocidad.	—	—	Enciende el pin PA7 si la velocidad es 0 (lenta), PA6 si es 1 (media) y PA5 si es 2 (rápida). Utiliza HAL_GPIO_WritePin() para setear o resetear cada pin de forma atómica.
HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)	Manejar las interrupciones generadas por los botones.	GPIO_Pin: número de pin que generó la interrupción.	—	Si han pasado más de 200 ms desde la última interrupción, incrementa o decrementa

				current_speed_level según el pin (PC13 para aumentar, PB8 para disminuir). Actualiza current_delay usando speed_delays y llama a update_speed_leds(). Almacena el tiempo actual en last_interrupt_time.
Inicialización en main()	Configurar el sistema y variables.	—	—	Inicializa la HAL, el reloj y las GPIO. Configura current_speed_level a 0, current_delay a 1500 ms, state_index a 0 y enciende el LED indicador de velocidad inicial. Escribe el primer patrón de secuencia en los ocho pines de salida y almacena last_tick.
Lógica principal (while(1))	Generar la secuencia de LEDs en tiempo real.	—	—	En un bucle infinito, evalúa si HAL_GetTick() - last_tick $\geq$ current_delay. Si se cumple, actualiza last_tick, incrementa state_index (reiniciando al llegar a 10), obtiene el siguiente patrón de secuencia y actualiza los ocho pines de salida. De esta forma se crea una

				temporización no bloqueante.
--	--	--	--	------------------------------

Nota: tabla 2 que muestras las funciones que implementa el código

– Tabla de puertos GPIO utilizados

Tabla 3: puertos GPIO

Puerto	Pin	Modo	Uso	Observaciones
GPIOA	PA10	Salida digital (Output Push-Pull)	LED de la secuencia (bit 0)	Corresponde al pin D2 en muchas placas Nucleo.
GPIOB	PB3	Salida digital	LED de la secuencia (bit 1)	Pin D3; requiere resistencia limitadora.
GPIOB	PB5	Salida digital	LED de la secuencia (bit 2)	Pin D4.
GPIOB	PB4	Salida digital	LED de la secuencia (bit 3)	Pin D5.
GPIOB	PB10	Salida digital	LED de la secuencia (bit 4)	Pin D6; se conecta un LED con resistencia.
GPIOA	PA8	Salida digital	LED de la secuencia (bit 5)	Pin D7.
GPIOA	PA9	Salida digital	LED de la secuencia (bit 6)	Pin D8.
GPIOC	PC7	Salida digital	LED de la secuencia (bit 7)	Pin D9.
GPIOA	PA7	Salida digital	LED indicador de velocidad lenta (nivel 0)	Pin D11.
GPIOA	PA6	Salida digital	LED indicador de velocidad media (nivel 1)	Pin D12.
GPIOA	PA5	Salida digital	LED indicador de velocidad rápida (nivel 2)	Pin D13.

Nota: puestas GPIO que se utilizaron en el proyecto

- Tabla de registros

Tabla 4: registros del código

Registro	Dirección (aprox.)*	Función	Bits relevantes	Relación con el proyecto
GPIOA_MODER	0x40020000	Configura el modo de los pines (entrada/salida)	PA5–PA10	Define los pines como salidas para LEDs
GPIOB_MODER	0x40020400	Configura el modo de los pines	PB3, PB4, PB5, PB10	Salidas digitales para LEDs
GPIOC_MODER	0x40020800	Configura el modo de los pines	PC7, PC13	PC7 salida (LED), PC13 entrada (botón)
GPIOA_ODR	0x40020014	Registro de datos de salida	PA5–PA10	Controla encendido/apagado de LEDs
GPIOB_ODR	0x40020414	Registro de salida	PB3, PB4, PB5, PB10	Salida de LEDs
GPIOC_ODR	0x40020814	Registro de salida	PC7	LED adicional
GPIOA_BSRR	0x40020018	Set/Reset de bits (salida segura)	PA5–PA10	Usado por HAL_GPIO_WritePin()
GPIOB_BSRR	0x40020418	Set/Reset de bits	PB3–PB10	Control de LEDs
GPIOC_BSRR	0x40020818	Set/Reset de bits	PC7	Control de LED
EXTI_IMR	0x40013C00	Máscara de interrupciones	Línea 8 y 13	Habilita interrupciones de botones
EXTI_RTSR	0x40013C08	Selección flanco ascendente	Línea 8 y 13	Detecta pulsación de botón
SYSCFG_EXTICR	0x40013808	Asocia pines a EXTI	PB8, PC13	Configura interrupciones externas
NVIC_ISER	0xE000E100	Habilita interrupciones en CPU	EXTI9_5, EXTI15_10	Permite ejecutar interrupciones
SysTick_CTRL	0xE000E010	Control del temporizador SysTick	ENABLE, TICKINT	Base de tiempo del sistema
SysTick_VAL	0xE000E018	Valor actual del contador	—	Base para HAL_GetTick()

Nota: Estos registros son configurados indirectamente mediante la biblioteca HAL de STM32

– Análisis de fallos

Tabla 5: fallos

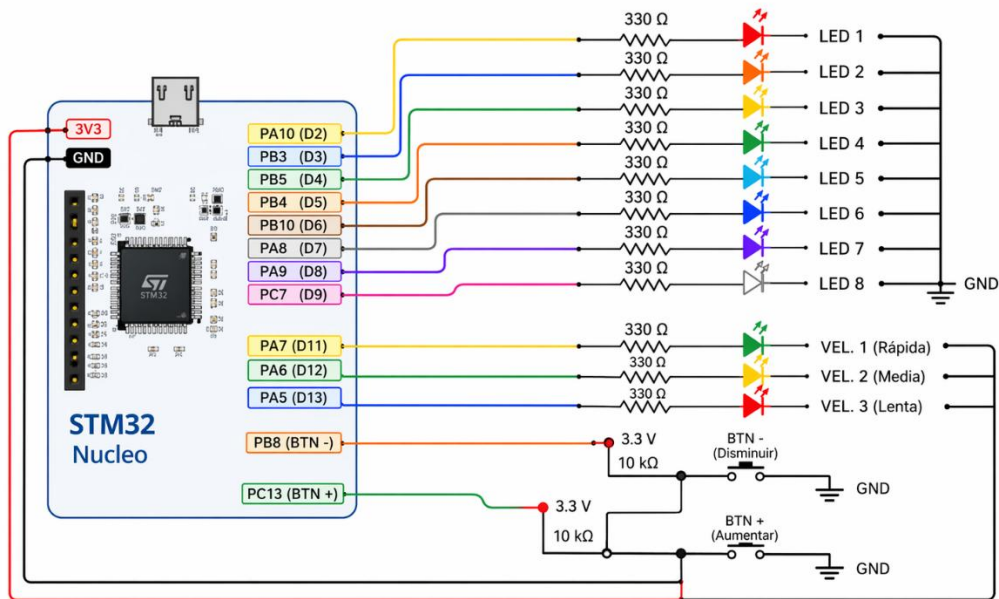
Problema	Causa probable	Cómo se detectó	Solución aplicada
Varias pulsaciones de un botón se registran como múltiples cambios de velocidad	Rebote mecánico del interruptor provoca múltiples interrupciones en un corto intervalo. Al presionar el botón, los contactos vibran generando varias transiciones de flanco.	Durante las pruebas, al presionar un botón la velocidad saltaba varios niveles de una sola vez.	Se implementó un <i>debounce</i> por software en HAL_GPIO_EXTI_Callback() comparando HAL_GetTick() con el tiempo de la última interrupción y rechazando eventos ocurridos dentro de los siguientes 200 ms.
Algunos LEDs de la secuencia no se encienden o parpadean inesperadamente	Errores en el cableado: conexiones incorrectas entre los pines del microcontrolador y los LEDs; falta de resistencias limitadoras; LEDs defectuosos.	Durante la construcción del circuito, se observó que ciertos patrones no coincidían con el diseño.	Se revisó la asignación de pines según el código (PA10, PB3, PB5, PB4, PB10, PA8, PA9, PC7) y se verificó la continuidad con un multímetro. Se conectaron resistencias de $\sim 220\ \Omega$ en serie con cada LED y se reemplazaron los diodos defectuosos.
El botón de disminuir velocidad no responde	Falta de resistencia pull-up o error en la configuración del pin PB8; el pin queda flotante y no detecta cambios de nivel.	Al presionar el botón de decremento, la velocidad no cambiaba.	Se activó la resistencia interna PULLUP para PB8 en la función MX_GPIO_Init() y se comprobó que la interrupción estuviera habilitada en HAL_NVIC_EnableIRQ(EXTI9_5_IRQn). También se verificó que el botón estuviera conectado entre PB8 y GND.

Nota: fallo que se vieron en el proyecto

– Evidencia de funcionamiento

6: Evidencia:

https://drive.google.com/file/d/1aIAfy_eZmcstuYUT8jNGBLPuUJtQ_75u/view?usp=sharing



Los 8 LEDs principales están conectados a pines del microcontrolador, y se encienden o apagan según una secuencia que está programada. Eso es lo que crea el efecto visual.

La velocidad de esa animación se puede cambiar con dos botones:

- uno aumenta la velocidad
- otro la disminuye

El microcontrolador detecta cuando presionas los botones usando interrupciones, así responde rápido sin estar revisando todo el tiempo.

También hay 3 LEDs extra que indican la velocidad actual: lento, medio o rápido.

En general, el sistema está en un ciclo que va cambiando los LEDs según el tiempo y la velocidad seleccionada.

– Asignación de roles

Tabla 7: roles

Integrante	Rol y actividades realizadas
Daniel Rivas	Encargado de la implementación del código en lenguaje C, desarrollando la lógica de la secuencia de LEDs, el control de velocidad y el manejo de botones mediante interrupciones (EXTI) con debounce. También en la elaboración del informe técnico en formato PDF.
Clever	Se encargó del diseño y construcción del circuito, incluyendo la conexión de los ocho LEDs, las resistencias, los botones y los LEDs indicadores. Elaboró los diagramas de conexión y de flujo, realizó la captura de evidencias

Nota: Roles de cada integrante

– Conclusiones

- Uso eficiente de la HAL y del contador SysTick. Implementar la temporización con HAL_GetTick() permitió crear un sistema no bloqueante: el microcontrolador continúa ejecutando código y atendiendo interrupciones mientras espera el siguiente evento. Este enfoque es esencial en sistemas embebidos donde se requieren múltiples tareas concurrentes.
- Ventajas de las interrupciones para la interacción usuario-sistema. Configurar los botones como entradas con interrupciones EXTI hace que el sistema responda inmediatamente a las pulsaciones sin necesidad de sondear constantemente el estado de los pines. Esto mejora la eficiencia y reduce el consumo de CPU, pero requiere implementar debounce para evitar múltiples activaciones por rebote.
- Importancia de la asignación correcta de pines. Durante el montaje del circuito se observó que una asignación incorrecta puede causar fallas en el encendido de los LEDs. Es indispensable consultar el esquema de la placa Nucleo y el código para asegurarse de que

cada LED y botón estén conectados a los pines adecuados y que se utilicen resistencias limitadoras para proteger los diodos.

– Bibliografía

8.4.1 GPIO port mode register (GPIOx_MODER) (x = A..I/J/K). (n.d.). Zhaw.Ch. Retrieved April 21, 2026, from https://ennis.zhaw.ch/wiki/lib/exe/fetch.php?media=stm32:register_descriptions:gpio_registers_st_rm.pdf#:~:text=8,where%20X%20means%20undefined

STM32F439xx HAL User Manual: IO operation functions. (n.d.). Upv.Es. Retrieved April 21, 2026, from https://www.disca.upv.es/aperles/arm_cortex_m3/livre/st/STM32F439xx_User_Manual/group__gpio__exported__functions__group2.html

(N.d.). Sciencedirect.com. Retrieved April 21, 2026, from <https://www.sciencedirect.com/topics/engineering/systick-timer#:~:text=All%20of%20the%20Cortex,as%20a%20simple%20timer%20peripheral>